CST44 – DESIGN AND ANALYSIS OF ALGORITHM

UNIT – 1

Algorithms: Definitions and notations: standard notations - asymptotic notations – worst case, best case and average case analysis; big oh, small oh, omega and theta notations; Recursive algorithms, analysis of non-recursive and recursive algorithms, solving recurrence equations, analyzing control structures.
Analysis of Sorting and Searching: Heap, shell, radix, insertion, selection and bubble sort; sequential, binary and Fibonacci search.

ALGORITHM:

- Algorithm is a sequence of unambiguous instructions to solve a problem.
- Algorithm is called as a solution for a given problem.
- A problem may have many solutions/algorithm. We need to select the best algorithm which is suitable for a given problem.
- After the algorithm for the problem is chosen, it is converted to program.

PROGRAM:

- Algorithm + Data Structure = Program
- The program may take zero or more input, process the instructions and produce a single output.

Characteristic of an Algorithm:

1. **Input:** Algorithm may take zero or more input
2. **Output:** Algorithm always produces a single output
3. **Definiteness:** Every instruction in an algorithm must be clear and unambiguous
4. **Finiteness :** For all possible inputs, algorithm must produce result in finite steps
5. **Effectiveness:** The algorithm must be feasible

Parameters considered to chose a best algorithm:

- Execution speed => Time taken to execute the algorithm for a specific input
- Memory => The memory required to store program and data.
- Easy to understand
- Easy to implement
- Correctness

ALGORITHMIC NOTATIONS:

Name of the algorithm : Each algorithm is given by its name
Data required for the algorithm: Data required for executing an algorithm is given as arguments to the algorithm
Comments : It describes the data passed to the algorithm or the steps of an algorithm

Steps of the algorithm: The sequence of steps which is performed by the algorithm

Read => Used to read the values

Write => Used to display the values

Example 1: Algorithm to add two numbers

```
Add(a,b)
c = a + b
write  c
```

In this example, add is a algorithm name, a and b are the data required to perform addition operation and the c= a+b and write c are the sequence of steps to perform addition operation.

Example 2: Algorithm to find greatest of two numbers

```
Great(a,b)
if(a > b)
then
write(" a is greatest")
else
write("b is greatest")
end if
```

Example 3: Algorithm to display n elements

```
Display(n)
for i = 1 to n
        write  i
end for
```

Example 4: Algorithm to perform sum of n elements

```
sum( n )
sum =0
for i = 1 to n
        sum = sum + i
end for
write  sum
```

Example 5: Algorithm to find maximum element in a given array

maxarray(a, n)

```
// a is an array of n elements
max = a[1]
for i = 2 to n
    if ( a[ i ] > max )
        max = a[ i ]
    end if
end for
write  max
```

- In the above algorithm, At the initial step, a[1] is assigned as max. In for loop, Each of the next element starting from second element is compared with max. If the element is greater than max, value of max changes. After comparing all the elements in array, value in max is displayed.

CREATING PROGRAMS

We consider five phases: requirements, design, analysis, coding, and verification.

(i) ***Requirements*:**

- Define and ***understand the objective/scope*** of the problem given.
- Based on objective, identify all the requirements to solve the problem.
- (i.e) What are the possible **inputs** for a given problem. What will be the **output** for a various inputs.

(ii) ***Design* :**

- Perform the design based on the requirements.
- The design which is developed should be simple and should specify entire aspects of the given problem.

(iii) ***Analysis*:**

- Analysis is performed to check whether the design which is developed is based on the specifications of the problem.
- If there is any deviations in the design, it is corrected and design is modified based on the problem specification.

(iv) Refinement and coding:

- After the design is made, the appropriate best algorithm is chosen for the problem.
- Select the programming language which is best suited for solving the problem. After selecting a programming language, coding is done for the chosen algorithm.

(v) Verification.

- Verification consists of three distinct aspects: Program proving, Testing and Debugging.
- ***Program proving :*** Before executing your program you should attempt to prove it is correct.
- ***Testing:*** The program is tested for all possible inputs, to find whether any faults/bug is available in a program or the program is giving wrong output. If the program fails to respond correctly then debugging is needed to determine what went wrong and how to correct it.
- ***Debugging:*** Correction of bug in a program, if found. After the bug is identified, steps need to be performed to correct the bug in order to produce the required output.

vi) Documentation:

- After the program is successfully developed, the entire description about the working of program is given in documentation for easy understanding.

ANALYZING PROGRAMS:

Several criteria to analyze the programs are

i) Does it do what we want it to do ? Results obtained according to the specification of the program

ii) Whether Documentation is present ? To describe how it works or how to use it

iii) Modularity exists ? Does the larger problem is divided into logically related sub-modules which will improve efficiency

iv) Is the program readable? Readability

There are other criteria for analyzing the programs which will have direct relationship with performance(i.e) based on the computation time and storage requirements of the algorithm.

Time complexity:

It is the amount of time required to execute the algorithm for the specific input.

Space complexity:

It is the amount of memory taken by the algorithm for execution.

Performance analysis (or) Asymptotic analysis:

The following information must be known to determine, how much time it takes to execute any command.

i) The machine executing on

ii) The machine language instruction set

ii) Time required by each machine instruction

Consider three examples

…….. …….. x = x + 1 …….. ……...	for i = 1 to n x = x +1 end for	for i = 1 to n for j = 1 to n x = x +1 end for end for
In this example, the statement x=x+1 is not inside any loop. Hence, Number of time the statement executed is 1 **Frequency count = 1**	In this example, the statement x=x+1 is inside for loop. Hence, Number of time the statement executed is n **Frequency count = n**	In nested for loop, Number of times statement executed is equal to the number of times the innermost loop is executed. In this example, the number of times the statement x=x+1 executed is n^2 **Frequency count = n^2**

Asymptotic notations:

- It allows us to analyze an algorithm's running time by identifying its behavior, as the input size for the algorithm increases. This is also known as an algorithm's growth rate.

i) Big Oh(O):

Big Oh is defined as

The function f(n) = O(g(n)) if and only if there exists constants c and n_0, such that

$f(n) \leq c * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm

Example:
$f(n) = 3n + 2 \leq 4*n$ for all $n \geq 2$, where $f(n) = 3n+2$, $c = 4$, $g(n) = n$ and $n_0 = 2$
$f(n) = O(1)$ represents computing time is constant
$f(n) = O(n)$ represents computing time is linear
$f(n) = O(n^2)$ represents computing time is quadratic
$f(n) = O(n^3)$ represents computing time is cubic
$f(n) = O(2^n)$ represents computing time is exponential
$f(n) = O(\log n)$ represents computing time is logarithmic

Note:
$O(1)<O(\log n)<O(n)<O(n^2)$
Here, Time taken to execute the algorithm having constant time O(1) is least. Time taken to execute the algorithm having quadratic time $O(n^2)$is high.

ii) Omega(Ω):
Omega is defined as
The function $f(n) = \Omega(g(n))$ if and only if there exists constants c and n_0, such that
$f(n) \geq c * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm
Example:
$f(n) = 3n + 2 \geq 3*n$ for all $n \geq 1$, where $f(n) = 3n+2$, $c = 3$, $g(n) = n$ and $n_0 = 1$

iii) Theta(Θ)
Theta is defined as
The function $f(n) = \Theta(g(n))$ if and only if there exists constants c1, c2 and n_0, such that
$c1 * g(n) \leq f(n) \leq c2 * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm
Example:
$3*n \leq 3n + 2 \leq 4*n$ for all $n \geq 2$, where $c1=3$, $c2=4$, $g(n) = n$, $f(n) = 3n+2$, $n_0 = 2$

<u>SOLVING RECURRENCES :-(Happen again (or) repeatedly)</u>

The indispensable last step when analyzing an algorithm is often to solve a recurrence equation. With a little experience and intention, most recurrence can be solved by intelligent guesswork. However, there exists a powerful technique that can be used to solve certain classes of recurrence almost automatically.

1. **Intelligent guess work:**
   This approach generally proceeds in 4 stages.
   1. Calculate the first few values of the recurrence
   2. Look for regularity.
   3. Guess a suitable general form.
   4. And finally prove by mathematical induction(perhaps constructive induction).

Then this form is correct.

Consider the following recurrence,

$$T(n) = \begin{cases} 0 & \text{if } n=0 \\ 3T(n \div 2)+n & \text{otherwise} \end{cases}$$

- First step is to replace n ÷ 2 by n/2
- It is tempting to restrict 'n' to being ever since in that case n÷2 = n/2, but recursively dividing an even no. by 2, may produce an odd no. larger than 1.
- Therefore, it is a better idea to restrict 'n' to being an exact power of 2.
- First, we tabulate the value of the recurrence on the first few powers of 2.

n	1	2	4	8	16	32
T(n)	1	5	19	65	211	665

* For instance, T(16) = 3 * T(8) +16
= 3 * 65 +16
= 211.

* Instead of writing T(2) = 5, it is more useful to write T(2) = 3 * 1 +2.

Then,
T(A) = 3 * T(2) +4
= 3 * (3 * 1 +2) +4
= $(3^2 * 1) + (3 * 2) +4$

* We continue in this way, writing 'n' as an explicit power of 2.

n	**T(n)**
1	1
2	3 * 1 +2
2^2	$3^2 * 1 + 3 * 2 + 2^2$
2^3	$3^3 * 1 + 3^2 * 2 + 3 * 2^2 + 2^3$
2^4	$3^4 * 1 + 3^3 * 2 + 3^2 * 2^2 + 3 * 2^3 + 2^4$
2^5	$3^5 * 1 + 3^4 * 2 + 3^3 * 2^2 + 3^2 * 2^3 + 3 * 2^4 + 2^5$

- The pattern is now obvious.

$T(2^k) = 3^k2^0 + 3^{k-1}2^1 + 3^{k-2}2^2+\ldots+3^12^{k-1} + 3^02^k$.

$= \sum 3^{k-i} 2^i$

$= 3^k \sum (2/3)^i$

$= 3^k * [(1 - (2/3)^{k+1}) / (1 - (2/3)]$
$= 3^{k+1} - 2^{k+1}$

Proposition: (Geometric Series)

Let S_n be the sum of the first n terms of the geometric series a, ar, ar^2….Then $S_n = a(1-r^n)/(1-r)$, except in the special case when r = 1; when $S_n = a_n$.

$= 3^k * [(1 - (2/3)^{k+1}) / (1 - (2/3))]$

$= 3^k * [((3^{k+1} - 2^{k+1})/ 3^{k+1}) / ((3 - 2) / 3)]$

$$= 3^k * \frac{3^{k+1} - 2^{k+1}}{3^{k+1}} * \frac{3}{1}$$

$$= 3^k * \frac{3^{k+1} - 2^{k+}1}{3^{k+1-1}}$$

$= 3^{k+1} - 2^{k+1}$

* It is easy to check this formula against our earlier tabulation.

Eg : 2

$$t_n = \begin{cases} 0 & n=0 \\ 5 & n=1 \\ 3t_{n-1} + 4t_{n-2}, & \text{otherwise} \end{cases}$$

$t_n = 3t_{n-1} - 4t_{n-2} = 0$ → General function

Characteristics Polynomial, $x^2 - 3x - 4 = 0$
$(x - 4)(x + 1) = 0$

Roots $r_1 = 4$, $r_2 = -1$

General Solution, $f_n = C_1 r_1^n + C_2 r_2^n$ → (A)
n=0 → $C_1 + C_2 = 0$ → (1)
n=1 → $C_1 r_1 + C_2 r_2 = 5$ → (2)

Eqn 1 → $C_1 = -C_2$

sub C_1 value in Eqn (2)
$-C_2 r_1 + C_2 r_2 = 5$

$C_2(r_2 - r_1) = 5$

$$C_2 = \frac{5}{r_2 - r_1}$$

$$= \frac{5}{-1 + 4}$$

$$= 5 / (-5) = -1$$

$C_2 = -1$, $C_1 = 1$

Sub C_1, C_2, r_1 & r_2 value in equation → (A)

$f_n = 1.\ 4^n + (-1)\ .\ (-1)^n$

$\mathbf{f_n = 4^n + 1^n}$

2.Homogenous Recurrences :

* We begin our study of the technique of the characteristic equation with the resolution of homogenous linear recurrences with constant co-efficient, i.e the recurrences of the form,

$a_0t_n + a_1t_{n-1} + \ldots.. + a_kt_{n-k} = 0$

where the t_i are the values we are looking for.

* The values of t_i on 'K' values of i (Usually $0 \le i \le k-1$ (or) $0 \le i \le k$) are needed to determine the sequence.

* The initial condition will be considered later.

* The equation typically has infinitely many solution.

* The recurrence is,

→ linear because it does not contain terms of the form t_{n-i}, t_{n-j}, t^2_{n-i}, and soon.

→ homogeneous because the linear combination of the t_{n-i} is equal to zero.

→ With constant co-efficient because the a_i are constants

* Consider for instance our non familiar recurrence for the Fibonacci sequence,

$f_n = f_{n-1} + f_{n-2}$

* This recurrence easily fits the mould of equation after obvious rewriting.

$f_n - f_{n-1} - f_{n-2} = 0$

* Therefore, the fibonacci sequence corresponds to a homogenous linear recurrence with constant co-efficient with $k=2, a_0=1 \& a_1=a_2 = -1$.

* In other words, if f_n & g_n satisfy equation.

So $\sum_{i=0}^{k} a_i f_{n-i} = 0$ & similarly for g_n & f_n

We set $t_n = C f_n + d g_n$ for arbitrary constants C & d, then t_n is also a solution to equation.

* This is true because,

$$\begin{aligned} a_0t_n + a_1t_{n-1} + \ldots + a_kt_{n-k} &= a_0(C f_n + d g_n) + a_1(C f_{n-1} + d g_{n-1}) + \ldots + a_k(C f_{n-k} + d g_{n-k}) \\ &= C(a_0 f_n + a_1 f_{n-1} + \ldots + a_k f_{n-k}) + d(a_0 g_n + a_1 g_{n-1} + \ldots + a_k g_{n-k}) \\ &= C * 0 + d * 0 \\ &= 0. \end{aligned}$$

1) (Fibonacci) Consider the recurrence.

$$f_n = \begin{cases} n & \text{if } n=0 \text{ or } n=1 \\ f_{n-1} + f_{n-2} & \text{otherwise} \end{cases}$$

We rewrite the recurrence as,

$$f_n - f_{n-1} - f_{n-2} = 0.$$

The characteristic polynomial is,

$$x^2 - x - 1 = 0.$$

The roots are,

$$x = \frac{-(-1) \pm \sqrt{((-1)2 + 4)}}{2}$$

$$= \frac{1 \pm \sqrt{(1 + 4)}}{2}$$

$$= \frac{1 \pm \sqrt{5}}{2}$$

$$r_1 = \frac{1 + \sqrt{5}}{2} \quad \text{and} \quad r_2 = \frac{1 - \sqrt{5}}{2}$$

The general solution is,

$$f_n = C_1r_1^n + C_2r_2^n$$

when n=0, $f_0 = C_1 + C_2 = 0$

when n=1, $f_1 = C_1r_1 + C_2r_2 = 1$

$C_1 + C_2 = 0 \rightarrow (1)$
$C_1r_1 + C_2r_2 = 1 \rightarrow (2)$

From equation (1)

$$C_1 = -C_2$$

Substitute C_1 in equation(2)

$$-C_2r_1 + C_2r_2 = 1$$
$$C_2[r_2 - r_1] = 1$$

Substitute r_1 and r_2 values

$$C_2 \frac{1 - \sqrt{5}}{2} - \frac{1 - \sqrt{5}}{2} = 1$$

$$C_2 \frac{1 - \sqrt{5} - 1 - \sqrt{5}}{2} = 1$$

$$\frac{-C_2 * 2\sqrt{5}}{2} = 1$$

$$-\sqrt{5}C_2 = 1$$

$C_1 = 1/\sqrt{5}$ $C_2 = -1/\sqrt{5}$

Thus,

$$f_n = \frac{1}{\sqrt{5}} \frac{1 + \sqrt{5}}{2}^n + \frac{-1}{\sqrt{5}} \frac{1 - \sqrt{5}}{2}^n$$

$$= \frac{1}{\sqrt{5}} \frac{1 + \sqrt{5}}{2}^n - \frac{1 - \sqrt{5}}{2}^n$$

3. Inhomogeneous recurrence :

* The solution of a linear recurrence with constant co-efficient becomes more difficult when the recurrence is not homogeneous, that is when the linear combination is not equal to zero.

* Consider the following recurrence

$$a_0t_n + a_1 t_{n-1} + \ldots + a_k t_{n-k} = b^n p(n)$$

* The left hand side is the same as before,(homogeneous) but on the right-hand side

we have $b^n p(n)$, where,

→ b is a constant

→ p(n) is a polynomial in 'n' of degree 'd'.

Example(1) :

Consider the recurrence,

$$t_n - 2t_{n-1} = 3^n \rightarrow (A)$$

In this case, b=3, p(n) = 1, degree = 0.

The characteristic polynomial is,

$$(x - 2)(x - 3) = 0$$

The roots are, $r_1 = 2$, $r_2 = 3$

The general solution,

$$t_n = C_1 r_1^n + C_2 r_2^n$$

$$t_n = C_1 2^n + C_2 3^n \rightarrow (1)$$

when n=0, $C_1 + C_2 = t0 \rightarrow (2)$

when n=1, $2C_1 + 3C_2 = t1 \rightarrow (3)$

sub n=1 in eqn (A)

$t_1 - 2t_0 = 3$

$t_1 = 3 + 2t_0$

substitute t_1 in eqn(3),

(2) * 2 ➔ $2C_1 + 2C_2 = 2t_0$

$2C_1 + 3C_2 = (3 + 2t_0)$

$-C_2 = -3$

$C_2 = 3$

Sub $C_2 = 3$ in eqn (2)

$C_1 + C_2 = t_0$

$C_1 + 3 = t_0$

$C_1 = t_0 - 3$

Therefore $t_n = (t_0-3)2^n + 3.\ 3^n$

$= \text{Max}[O[(t_0 - 3)\ 2^n], O[3.3^n]]$

$= \text{Max}[O(2^n), O(3^n)]$ constants

$\mathbf{= O[3^n]}$

4. Change of variables:

* It is sometimes possible to solve more complicated recurrences by making a change of variable.

*In the following example, we write T(n) for the term of a general recurrences, and t_i for the term of a

new recurrence obtained from the first by a change of variable.

Example: (1)
Consider the recurrence,

$$T(n) = \begin{cases} 1 & , \text{ if } n=1 \\ 3T(n/2) + n & , \text{ if 'n' is a power of 2, } n>1 \end{cases}$$

➔ Reconsider the recurrence we solved by intelligent guesswork in the previous section, but only for the case when 'n' is a power of 2

$$T(n) = \begin{cases} 1 \\ 3T(n/2) + n \end{cases}$$

* We replace 'n' by 2^i.
* This is achieved by introducing new recurrence t_i, define by $t_i = T(2^i)$
* This transformation is useful because n/2 becomes $(2^i)/2 = 2^{i-1}$
* In other words, our original recurrence in which T(n) is defined as a function of T(n/2) given way to one in which t_i is defined as a function of t_{i-1}, precisely the type of recurrence we have learned to solve.

$t_i = T(2^i) = 3T(2^{i-1}) + 2^i$
$t_i = 3t_{i-1} + 2^i$
$t_i - 3t_{i-1} = 2^i$ →(A)

In this case,
b = 2, p(n) = 1, degree = 0

So, the characteristic equation,
$(x - 3)(x - 2) = 0$

The roots are, r1 = 3, r2 = 2.

The general equation,
$t_n = C_1 r_1^i + C_2 r_2^i$
sub. r_1 & r_2: $t_n = 3^n C_1 + C_2 2^n$
$t_n = C_1 3^i + C_2 2^i$

We use the fact that, $T(2^i) = t_i$ & thus T(n) = tlogn when $n= 2^i$ to obtain,
$T(n) = C_1 . 3^{\log_2 n} + C_2 . 2^{\log_2 n}$
$T(n) = C_1 . n^{\log_2 3} + C_2.n$ [i = logn]
When 'n' is a power of 2, which is sufficient to conclude that,

$T(n) = O(n^{\log 3})$ 'n' is a power of 2

Example: (2)

Consider the recurrence,
$T(n) = 4T(n/2) + n^2$ → (A)

Where 'n' is a power of 2, $n \geq 2$.

$t_i = T(2^i) = 4T(2^{i-1}) + (2^i)^2$

$t_i = 4t_{i-1} + 4^i$

→ $t_i - 4t_{i-1} = 4^i$

In this eqn,
b = 4, P(n) = 1, degree = 0

The characteristic polynomial,
$(x - 4)(x - 4) = 0$

The roots are, $r_1 = 4$, $r_2 = 4$.

So, the general equation,
$t_i = C_1 4^i + C_2 4^i.i$ [since i = logn]
$= C_1\, 4^{\log n} + C_2.\, 4^{\log n}.\, \log n$ [since $2^i = n$]
$= C_1 .\, n^{\log 4} + C_2.\, n^{\log 4}.n^{\log 1}$

T(n) = $O(n^{\log 4})$ 'n' is the power of 2.

Example : 3

$T(n) = 2T(n/2) + n \log n$

When 'n' is a power of 2, $n \geq 2$
$t_i = T(2^i) = 2T(2^i/2) + 2^i .i$ [since $2^i = n$; i =logn]
$t_i - 2t_{i-1} = i.\, 2^i$

In this case,
b = 2, P(n) = i, degree = 1
$(x - 2)(x - 2)^2 = 0$

The roots are, $r_1 = 2$, $r_2 = 2$, $r_3 = 2$

The general solution is,

$t_n = C_1 2^i + C_2.\, 2^i .\, i + C_3.\, i^2.\, 2^i$

$= nC_1 + nC_2 + nC_3(\log n^2{}_2 n)$

$t_n = O(n.\log^2{}_2 n)$

Example: 4

$$T(n) = \begin{cases} 2 & , \quad n=1 \\ 5T(n/4) + Cn^2 & , \quad n>1 \end{cases}$$

$t_i = T(4^i) = 5T(4^i/4) + C(4^i)^2$
$= 5T\ 4^{\ i-1} + C.\ 16^i$
$= 5t_{\ i-1} + C.16^i$
$t_i - 5t_{\ i-1} = C.\ 16^i$

In this case,
b = 16, P(n) = 1, degree = 0

The characteristic eqn,
$(x - 5)(x - 16) = 0$

The roots are, $r_1 = 5$, $r_2 = 16$

The general solution,
$t_i = C_1.5^i + C_2.16^i$
$= C_1.5^i + C_2.(4^2)^i$
$\mathbf{t_n = O(n^2)}$

Example: 5

$$T(n) = \begin{cases} 2 & , n = 1 \\ T(n/2) + Cn & , n > 1 \end{cases}$$

$T(n) = T(n/2) + Cn$
$= T(2^i/2) + C.\ 2^i$
$= T(2^{\ i-1}) + C.\ 2^i$
$t_i = t_{\ i-1} + C.\ 2^i$
$t_i - t_{\ i-1} = C.\ 2^i$

In this case, b=2, P(n) =1, degree =0

So, the characteristic polynomial,
$(x -1)(x - 2) = 0$

The roots are, $r_1 = 1$, $r_2 = 2$
$t_i = C_1.\ 1^i + c_2.\ 2^i$
$= C_1.\ 1^{\log_2 n} + C_2.n$
$= C_1\ .\ n^{\log_2 1} + C_2.n$

t_n = O(n)

Example: 6

$$T(n) = \begin{cases} 1 & , n = 1 \\ 3T(n/2) + n; & n \text{ is a power of } 2 \end{cases}$$

$t_i = T(2^i) = 3T(2^i/2) + 2^i$
$= 3T(2^{i-1}) + 2^i$
$t_i = 3t_{i-1} + 2^i$

So, b = 2, P(n) =1, degree = 0
$(x - 3)(x - 2) = 0$

The roots are, $r_1 = 3$, $r_2 = 2$
$t_i = C_1. 3^i + C_2. 2^i$
$= C_1. 3^{\log_2 n} + C_2. 2^{\log_2 n}$
$= C_1. n^{\log_2 3} + C_2. n^{\log_2 2} = 1$
$= C_1. n^{\log_2 3} + C_2.n$
$t_n = O(n^{\log_2 3})$

Example: 7

$T(n) = 2T(n/2) + n. \log n$

$t_i = T(2^i) = 2T(2^i/2) + 2^i . i$
$= 2T(2^{i-1}) + i. 2^i$
$= 2t_{i-1} + i.2^i$
$t_i - 2t_{i-1} = i. 2i$

➔ b=2, P(n) = I, degree = 1
The roots is $(x - 2)(x - 2)2 = 0$
$x = 2,2,2$

General solution,
$t_n = C_1.r_1^i + C_2.i.r_2^i + C_3. i2. r_3^i$
$= C_1.2^i + C_2. i.2^i + C_3. i^2.2^i$
$= C_1. n + C_2.n.^{\log_2 n} + C_3.i^2.n$
$= C_1.n + C_2.n.^{\log_2 n} + C_3(2^{\log_2 n}).n$

$t_n = O(n. 2^{\log_2 n})$

5. Range Transformation:

* When we make a change of variable, we transform the domain of the recurrence.
* Instead it may be useful to transform the range to obtain a recurrence in a form that we know how to solve
* Both transformation can be sometimes be used together.

Example: 1

Consider the following recurrence , which defines T(n). where 'n' is the power Of 2

$$T(n) = \begin{cases} 1/3 & \text{, if } n=1 \\ n\,T^2(n/2) & \text{, otherwise} \end{cases}$$

The first step is a change of variable,

Let t_i denote $T(2^i)$

$t_i = T(2^i) = 2^i\, T^2(2^{\,i-1})$

$= 2^i\, t^2_{\,i-1}$

* This recurrence is not clear, furthermore the co-efficient 2^i is not a constant.
* To transform the range, we create another recurrence by using ui to denote lg t_i

$u_i = \lg t_i = i + 2\lg t_{\,i-1}$

$= i + 2u_{\,i-1}$

➔ $u_i - 2u_{\,i-1} = i$

$(x-2)(x-1)^2 = 0$

The roots are, x = 2, 1, 1.

G.S,

$u_i = C_1.2^i + C_2.1^i + C_3.i.1^i$

Sub. This solution into the recurrence,

For u_i yields,

$i = u_i - 2u_{\,i-1}$

$= C_1 2^i + C_2 + C_3.i - 2(C_1.\,2^{\,i-1} + C_2 + C_3\,(i-1))$

$= (2C_3 - C_2) - C_3 i.$

$C_3 = -1$ & $C_2 = 2C_3 = -2$

$u_i = C_1 2^i - i - 2$

This gives us the G.S for t_i & T(n)

$t_i = 2^{ui} = 2^{C_1 2i - i-2}$

$T(n) = t_{\,lgn} = 2^{C_1 n - logn - 2}$

$= 2^{C_1 n} / 4n$

We use he initial condition T(1) = 1/3

To determine C1: $T(1) = 2^{C_1} / 4 = 1/3$

Implies that C1 = lg(4/3) = 2 – log 3

The final solution is

$T(n) = 2^{\,2n} / 4n\, 3^n$

ARRAY:

- Array is a collection of elements of same data type that are stored under a common name.

Characteristics of array:

- The elements of array are stored in contiguous memory location.
- Individual element of array can be accessed by using array name followed by a integer enclosed with a square bracket which is called as subscript or index. Example: Second element in array is accessed by a[1].

Operations performed in an array:

i) Traversing => Used to visit all the elements in an array

ii) Sorting => Used to sort the elements of an array in ascending or descending order

iii) Searching => Used to search a given element from an array

iv) Insertion => Used to insert the element in the specified location in an array provided that array is not full

v) Deletion => Used to delete the particular element from array

vi) Merging => Used to merge the elements of two arrays into a single array

Algorithms for operations of array:

i) Traversing:

```
for i = 1 to n
visit( a[ i ] ) (or) display( a[ i ] )
end for
```

ii) Sorting:

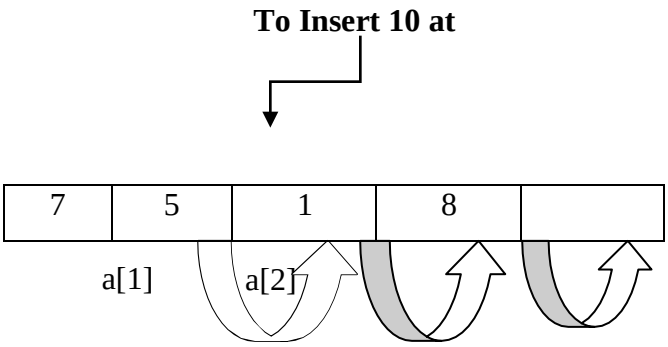
Write any sorting algorithm

iii) Searching:

Write any searching algorithm

iv) Insertion:

```
if  a[high] ! = NULL
location 2
    Write "Array is full, cant insert element"
else
i=high
   while  i>Location
a[5]
        a[ i ] = a[ i – 1 ]
```



```
        i - - ;
    end while
a[Location] = key
end  if
```

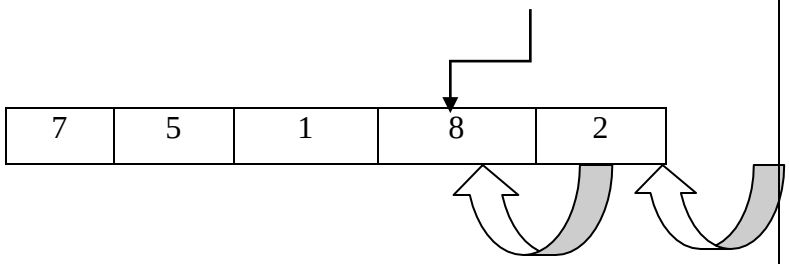
vi) Deletion:

Get the key element which is to be deleted
Initialize i=0
i = searcharray(a,key) // i gives the position of the key element

```
if i = = 0
    write " key not found"
```

To delete the element 1

```
else
     while i < high
          a[ i ] = a [ i+1 ]
          i + +
     end while
end  if
a[high] = NULL
high = high -1
```



vi) Merging

write the merge algorithm in merge sort

<u>SEARCHING:</u>

- Searching is a processing of finding a given element from a set of elements.
- Searching is successful if the element is found or else it is unsuccessful

<u>i) LINEAR SEARCH:</u>

- Linear search is performed either in sorted or unsorted list.
- It is a process of searching a given element from the array linearly one by one until the nth element in array.
- If the element is found, the position of that element is returned.
- If the element is not found in array, searching becomes unsuccessful.

ALGORITHM:

Linearsearch(A,n,x)

```
//A is an array which contains n elements
// x is a search element
flag=0
for i=1 to n
        if x= = a[ i ]
                write the position of x
                flag = 1
```

```
                break
        end if
end for
if flag = = 0
        Write element is not found
```

Example:

4	7	10	8	3
[1]	a[2]	a[3]	a[4]	a[5]

Here n = 5, x = 10(search element)

<u>Comparison 1:</u>

i = 1 , 1 < = 5 => True
x = = a[i] => 10 = = a[1]
10 = = 4 => False

<u>Comparison 2:</u>

i = 2 , 2 < = 5 => True
x = = a[i] => 10 = = a[2]
10 = = 7 => False

<u>Comparison 1:</u>

i = 3 , 3 < = 5 => True
x = = a[i] => 10 = = a[3]
10 = = 10 => True

Position of the search element 10 is displayed. (i.e) The position 3 is displayed

Complexity analysis for linear search:

Best case:

Search element matches with the first element

Number of comparisons: 1

Asymptotic complexity : **T(n) = O(1)**

Worst case:

Search element doesn't exist

Number of comparisons: n

Asymptotic complexity : **T(n) = O(n)**

Average case:

Search element may be present at any position in array

Expected number of comparisons T(n) = $T(n) = \sum_{i=1}^{n} Pi \, . \, i$

$P_i = \frac{1}{n}$

$T(n) = \frac{1}{n} * \frac{n(n+1)}{2}$

ii) $T(n) = \frac{n+1}{2}$

iii $T(n) = O(n)$

iv)

ii)BINARY SEARCH:

- Binary search algorithm is the ***fast searching algorithm***. It works on the principle of divide and conquer.
- To perform binary search, the ***input elements should be in sorted order***.

Binary Search algorithm:

```
BinarySearch(A,x,low,high)
// A is an array of n elements
// low =1 and high= n
while(low < = high)
        mid = (low + high) / 2
    if(x = = a[mid])
        return mid
   else if(x > a[mid])
        low = mid +1
   else
        high = mid -1
end while
```

- **Step1:** In binary search, Initially our search range is the entire array. Hence, low points to 1st position and high points to nth position.
- **Step2:** While the low < = high, find the position of the middle element. Compare of search element with the middle element. If it matches, its position is returned.
- **Step3:** If the search element is greater than the middle element, Search range will become right side of the middle element. Hence low is changed to mid+1. ***Goto step2***
- **Step4:** If the search element is less than the middle element, Search range will become left side of the middle element. Hence high is changed to mid-1. ***Goto step2***

Example:

2	4	6	8	10
a[1]	a[2]	a[3]	a[4]	a[5]

Here n = 5, x = 8(search element)

low = 1, high =5

while (1<=5)

mid = (1+5)/2 => ***mid =3***

2	4	6	8	10
a[1]	a[2]	a[3]	a[4]	a[5]
low		mid		high

if x = = a[mid] => if 8 = = a[3] => (i.e) 8 = = 6 =>False
else if x > a[mid] => 8 > a[3] => (i.e) 8 > 6 => True
Since search element 8 > middle element 6, search range will be on right side of middle element.
Hence, low = mid +1

2	4	6	8	10
a[1]	a[2]	a[3]	a[4]	a[5]
		mid	low	high

Now *low = 4 and high =5*
While 4 < =5
mid = (4+5) / 2 => **mid = 4**

2	4	6	8	10
a[1]	a[2]	a[3]	a[4]	a[5]
			mid low	high

if x = = a[mid] => 8 = = a[4] => 8 = = 8 => True
Since the element is found, the position of the search element is returned and displayed.
Complexity analysis for binary search:
Best case:
If the search element is present in the middle position, Number of comparison = 1
Asymptotic complexity : **T(n) = O(1)**
Worst case:
Number of comparisons = $\log_2 n$
Asymptotic complexity : **T(n) = O(logn)**
Average case:
Number of comparisons = $\log_2 n$
Asymptotic complexity : **T(n) = O(logn)**

v) FIBONACCI SEARCH

- The Fibonacci search technique is a method of searching a sorted array using divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers .
- Fibonacci search divides the array into two unequal parts that have sizes that are consecutive Fibonacci numbers

Algorithm:

Let a[1..n] be the input array and element to be searched be x.

1. Find the smallest Fibonacci Number greater than or equal n. Let this number be Fn. Let the two Fibonacci numbers preceding it be Fn-1 and Fn-2 respectively.
2. While the array has elements to be inspected:
   a. The position of the element which is to be compared is found by **i = min(offset +Fn-2 , n).** Initially offset value is set to zero. In subsequent executions, offset is the difference between values of old Fn and new Fn.
   b. Compare x with the element which is at position i. **If** x = = element, return that position.
   c. Else if **x > element**, the value of Fn will be the previous Fibonacci value from the current value. The corresponding Fn-1 and Fn-2 values are changed accordingly and value of i is found. The value in that position is compared. If it matches return that position.
   d. Else if **x < element**, the value of Fn will be the 2nd previous Fibonacci value from the current value. The corresponding Fn-1 and Fn-2 values are changed accordingly and value of i is found. The value in that position is compared. If it matches return that position.
   e. When there is a single element remaining for comparison, (i.e.,) if Fn-1 = 1, compare x with that remaining element. If match, return that position.
   f. Goto step c

Example:

2	4	6	8	10	12
a[1]	a[2]	a[3]	a[4]	a[5]	a[6]

Fibonacci series is 0, 1, 1, 2, 3,5,8,13,21,34……

Fibonacci number **Fn = Fn-1 + Fn-2**

Here n = 6, x = 8(search element)

The smallest Fibonacci number which is greater than n=6 is 8. **Fn =8**

Fn-1 = 5

Fn-2 = 3

Fn-2	**Fn-1**	**Fn**	**Offset**	**i=min(offset+Fn-2, n)**	**a[i]**	**Comparison**
3	5	**8**	0	i = min(0+3, 6) i=3	a[3] =6	8> 6 => Fn changed to prev Fibonacci value in next step
2	3	**5**	3	i = min(3+2, 6) i = 5	a[5]=10	8<10 => Fn changed to 2nd prev Fibonacci value in next step
1	1	**2**	3	i = min(3+1, 6) i = 4	a[4] =8	8 = = 8 => The position of search element 8 is returned.

SORTING:

- Sorting is a process of arranging the set of elements either in ascending or descending order.

i) BUBBLE SORT:

- In bubble sort, ***for n elements, there will be a maximum of n-1 passes*** to make a sorted list of elements. In each pass, each element is compared with its adjacent element in the array. If the kth element is greater than $k+1^{th}$ element, Swapping is performed on those two elements. By this way all the elements are compared.
- ***k is the number of comparison in each pass. In pass 1, Number of comparison =n-1, In pass 2, number of comparison = n-2*** and so on. *After the end of each pass,* the largest element will be settled at the actual position in array. Hence, *number of comparisons is reduced by 1 after each pass.*

Algorithm:

```
Bubblesort(a, n)
// a is an array of n elements
   for j = 1 to n-1
         for k= 1 to n – j
              if (a[k] > a[k+1] )
                     swap (a[k] , a[k+1] )
              end if
         end for
   end for
   write the array a
```

Example:

Pass 1: j=1

a[1]	a[2]	a[3]	a[4]	a[5]
6	4	8	5	2
a[1] > a[2] => 6 > 4 => True => Swap done				
4	6	8	5	2
a[2] > a[3] => 6>8 => False				
a[3] > a[4] => 8>5 => True => Swap done				
4	6	5	8	2
a[4] > a[5] => 8>2=> True => Swap done				
4	6	5	2	8

Pass 2: j=2

a[1]	a[2]	a[3]	a[4]	a[5]
4	6	5	2	8
a[1] > a[2] => 4>6 => False				
4	6	5	2	8

a[2] > a[3] => 6>5 = True => Swap done				
4	5	6	2	8
a[3] > a[4] => 6>2 => True => Swap done				
4	5	2	6	8

Pass 3: j=3

a[1]	a[2]	a[3]	a[4]	a[5]
4	5	2	6	8
a[1] > a[2] => 4>5 => False				
4	5	2	6	8
a[2] > a[3] => 5>2 = True => Swap done				
4	2	5	6	8

Pass 4: j=4

a[1]	a[2]	a[3]	a[4]	a[5]
4	2	5	6	8
a[1] > a[2] => 4>2 => True=> Swap done				
2	4	5	6	8

Elements in sorted order:

a[1]	a[2]	a[3]	a[4]	a[5]
2	4	5	6	8

Analysis:

- The number of comparisons made

  $1 + 2 + 3 + \ldots + (n - 1) = n(n - 1)/2 = O(n^2)$
- In this algorithm the number of comparison is irrespective of data set i.e., input whether best or worst.
- Time Complexity $T(n) = O(n^2)$
- It does not require extra memory.

ii) Selection sort:

- Selection sort algorithm selects the smallest element in array and exchange it with the 1st element. From the remaining set of elements, selects the smallest element and exchange it with 2nd element and so on.
- Initially k is the position of the minimum element. The kth element is compared with the remaining elements in array and the position of actual minimum element is found. If that position is not equal to k, then swapping of those two elements is done. This process is continued from k = 1 to n-1. By this way, entire array is sorted.

Algorithm:
SELECTIONSORT(A,N)

```
// a is an array of n elements
for k = 1 to n -  1
        min = k
        for j = k +1 to n
            if( a[ j ] < a[ min ] )
                min =j
            end if
        end for
        if( min ! = k)
                Swap ( a[k] , a[min])
        end if
end for
```

Example:
k=1, min = 1

6	4	8	5	2
a[1]	a[2]	a[3]	a[4]	a[5]

↑ min

j=2 , a[2] < a[1] => 4 < 6 => True
min = 2
j=3, a[3] < a[2] => 8<4 => False
j=4, a[4] < a[2] => 5<4 => False
j=5 , a[5] < a[2] => 2<4 => True
min =5
since 1 ! = 5
swap a[1] and a[5]

2	4	8	5	6
a[1]	a[2]	a[3]	a[4]	a[5]

k=2, min =2

2	4	8	5	6
a[1]	a[2]	a[3]	a[4]	a[5]

↑ min

j=3, a[3] < a[2] => 8 < 4 => False
j=4, a[4] < a[2] => 5 < 4 => False
j=5, a[5] < a[2] => 6 < 4 => False
Since 2 = = 2 , Swap not performed

2	4	8	5	6
a[1]	a[2]	a[3]	a[4]	a[5]

k=3, min =3

2	4	8	5	6
a[1]	a[2]	a[3]	a[4]	a[5]

min

j=4, a[4] < a[3] => 5< 8 => True

min = 4

j=5, a[5[< a[4] => 6 < 5 => False

since 4 ! = 3

swap a[3] and a[4]

2	4	5	8	6
a[1]	a[2]	a[3]	a[4]	a[5]

k=4, min =4

2	4	5	8	6
a[1]	a[2]	a[3]	a[4]	a[5]

min

j=5, a[5] < a[4] => 6< 8 => True

min = 5

since 5 ! = 4

swap a[4] and a[5]

2	4	5	6	8
a[1]	a[2]	a[3]	a[4]	a[5]

Analysis:

The number of moves with this technique is always of the order O(n).So the time complexity is O(n).

iii) INSERTION SORT:

- In insertion sort, the key element is chosen and that key element will be inserted at the appropriate position by comparing all the elements to left of a key element.
- The first key element chosen will be 2nd element, next is 3rd element and it goes on upto n.(i.e) chosing a key element ranges from 2 to n.

Algorithm:

```
Insertionsort(a,n)
// A is an array of n elements
for  j= 2 to n     // j represents the position of key element
      key = a[j]
      i = j – 1
     while ( i> 0 and key < a[ i ] )
         a[ i + 1 ] = a[ i ]
         i = i – 1
     end while
```

```
        a[ i + 1] = key
    end for
    write  a
```

Example:

j=2, key = 4

i = 1

6	4	8	5
a[1]	a[2]	a[3]	a[4]

↑ i

while 1>0 and 4<6 => True

a[2]=a[1] => a[2] = 6

i=0

6	6	8	5
0 a[1]	a[2]	a[3]	a[4]

↑ i

while 0 > 0 => False

a[0+1] = 4.

4	6	8	5
a[1]	a[2]	a[3]	a[4]

j=3, key =8

i=2

while 2>0 and 8<6 => False

a[i+1] = key => a[3] = 8 => No changes here

j=4, key =5

i=3

4	6	8	5
a[1]	a[2]	a[3]	a[4]

↑ i

while 3>0 and key < a[3] => 5 < 8 => True

a[3+1]=a[3] => a[4]=8

4	6	8	8
a[1]	a[2]	a[3]	a[4]

i=2,

4	6	8	8

a[1]	a[2]	a[3]	a[4]
	↑ i		

while 2> 0 and 5 < a[2] => 5 < 6 => True

a[2+1] = a[2] => a[3]=6

4	6	6	8
a[1]	a[2]	a[3]	a[4]

i=1,

4	6	6	8
a[1]	a[2]	a[3]	a[4]
↑ i			

while 1>0 and 5< a[1] => 5< 4 => False

4	5	6	8
a[1]	a[2]	a[3]	a[4]

Analysis:

Best-Case:

The while-loop in line 5 executed only once for each j. This happens if given array A is already sorted.

T(n) = an + b = O(n)

T(n)=O(n)

It is a linear function of n.

Worst-Case:

The worst-case occurs, when line 5 executed j times for each j. This can happens if array A starts out in reverse order .

T(n) = an2 + bc + c = O(n2)

T(n)=O(n2)

It is a quadratic function of n.

Extra Memory

This algorithm does not require extra memory.

<u>iv)</u> SHELL SORT:

Step1: The current element element is compared with the another element which is at certain distance and swapping is performed, if necessary.

Step 2: If swapping is not performed with a current distance, distance is reduced by half and step 1 continues until distance becomes zero.

Algorithm:

Shellsort(a , n)

```
// a is an array of n elements
        dist=n/2
        repeat
                repeat
                        swap=0
```

```
                    for i=1 to n-dist
                        if(a[i]>a[i+dist])
                            swap( a[i] , a[i+dist])
                            swap=1
                        end if
                    end for
                until swap=1
            dist = dist/2
            until dist!=0
write a
```

Example:

6	4	8	5	2
a[1]	a[2]	a[3]	a[4]	a[5]

dist = 5/2 **=> dist = 2**

swap=0

i =1

6	4	8	5	2
a[1]	a[2]	a[3]	a[4]	a[5]

a[1] > a[3] => False

i =2

6	4	8	5	2
a[1]	a[2]	a[3]	a[4]	a[5]

a[2] > a[4] => False

i=3

6	4	8	5	2
a[1]	a[2]	a[3]	a[4]	a[5]

a[3] > a[5] => True => Swap a[3] and a[5] , ***swap =1***

6	4	2	5	8
a[1]	a[2]	a[3]	a[4]	a[5]

while(swap) => while(1)
Set swap =0
i =1

6	4	2	5	8
a[1]	a[2]	a[3]	a[4]	a[5]

a[1] > a[3] => True => Swap a[1] and a[3], ***swap=1***

2	4	6	5	8
a[1]	a[2]	a[3]	a[4]	a[5]

i=2 and i=3

2	4	6	5	8
a[1]	a[2]	a[3]	a[4]	a[5]

a[2] > a[4] => False a[3]> a[5] => False

While(swap) => while(1)
Set swap=0

i =1, i=2 and i=3

2	4	6	5	8
a[1]	a[2]	a[3]	a[4]	a[5]

a[1]> a[3]=>F

a[2] > a[4] => F a[3]> a[5] => F

while(swap) => while(0)
dist= 2/2 => **dist = 1**
while(distance) => while(1)
swap=0
i=1,i=2, i=3

2	4	6	5	8

a[1] a[2] a[3] a[4] a[5]

a[1]>a[2] =>F a[2]>a[3] =>F a[3]> a[4] => True => *Swap a[3] and a[4]* , ***swap =1***

2	4	5	6	8
a[1]	a[2]	a[3]	a[4]	a[5]

while(swap) => while(1)
swap =0
i=1, i=2, i=3, i=4

2	4	5	6	8
a[1]	a[2]	a[3]	a[4]	a[5]

a[1]>a[2]=>F a[2]>a[3]=>F a[3]>a[4]=>F a[4] > a[5] => F
while(swap) => while(0) => False
dist = 1/2 => **dist = 0**
while(dist) => while(0) => False

Sorted elements:

2	4	5	6	8
a[1]	a[2]	a[3]	a[4]	a[5]

vii) RADIX SORT

- Radix sort is an non-comparative sorting algorithm
- Radix sort is also called as bucket sort

Algorithm:

```
Find the maximum element in the array
Find the number of digits in that element, say w
for k = 1 to w // k represents the number of digits in that number
   for i = 1 to n
         L = (a[ i ] / pow(10, k-1) ) % 10
         Enqueue( Q[L] , a[ i ] )
    end for
    set j =1
    for i = 0 to 9
       while( Q[ i ] !=empty)
          a[ j++ ] = dequeue( Q[ i ] )
       end while
    end for
end for
```

Example:

Consider the array

150	213	002	041

a[1] a[2] a[3] a[4]

The maximum element in the array is 213.

Number of digits in this number is 3. (i.e) w=3

Q[0]	150			
Q[1]	041			
Q[2]	002			
Q[3]	213			
Q[4]				

.

.

Q[9]

Traceout:

i) Storing the elements in array to the bucket based on the unit digit.

k=1, 1<=3

i =1

L = (a[1] / pow(10,0)) % 10

L = 150 % 10 => **L =0**

Q[0]=150

i =2

L = 213 % 10 => **L =3**

Q[3]=213

i =3

L = 002 % 10 => **L =2**

Q[2]=002

i =4

L = 041 % 10 => **L =1**

Q[1]=041

Removing elements from bucket and store in array

150	041	002	213
a[1]	a[2]	a[3]	a[4]

ii) Storing the elements in array to the bucket based on the tenth digit.

002				Q[0]
213				Q[1]
				Q[2]
				Q[3]
041				Q[4]
150				Q[5]

.

.

Q[9]

k=2, 2<=3

i =1, a[1]=1**5**0
L = (a[1] / pow(10,1)) % 10
L = (150/10) % 10 => L = 15 % 10 => **L=5**
Q[5]=150
i =2, a[2]=0**4**1
L =4
Q[4]=041
i =3, a[3]=0**0**2
L = 0
Q[0]=002
i =4, a[4]=2**1**3
L =1
Q[1]=213

Removing elements from bucket and store in array

002	213	041	150
a[1]	a[2]	a[3]	a[4]

iii) Storing the elements in array to the bucket based on the hundredth digit.

002	041			Q[0]
150				Q[1]
213				Q[2]
				Q[3]
				Q[4]

.
.
Q[9]

k=3, 3<=3

i =1, a[1]=**0**02
L = (a[1] / pow(10,1)) % 10
L = (002/100) % 10 => L = 0 % 10 => **L=0**
Q[0]= 002
i =2, a[2]=**2**13
L =2
Q[2]=213
i =3, a[3]=**0**41
L = 0
Q[0]=041
i =4, a[4]=**1**50
L =1

Q[1]=150

Removing elements from bucket and store in array

002	041	150	213
a[1]	a[2]	a[3]	a[4]

viii)<u>HEAP SORT:</u>

A heap is a complete binary tree with the property that the value at each node is at least as large as the value at its children.

The definition of a max heap implies that one of the largest elements is at the root of the heap. If the elements are distinct then the root contains the largest item.The tree is completely filled on all levels except possibly lowest.

// CREATES HEAP

```
heapify(a,n)
        for i=n/2 to 1
                adjust(a,i,n);
        end for
```

// ADJUST THE HEAP TO SATISFY HEAP PROPERTY

```
adjust(a,i,n)
        j=2*i;
        key=a[i];
        while(j<=n)
                if((j<n)&&(a[j]<a[j+1]))
                        j=j+1
                end if
                if(key>a[j])
                        break
                end if
                a[j/2]=a[j]
                j=2*j
        end while
        a[j/2]=key
```

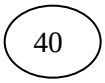
// PERFORMS SORTING OPERATION

```
hsort(a, n)
        heapify(a,n);
        for i=n to 2
                swap(a[i],a[1])
                adjust(a,1,i-1)
        end for
```

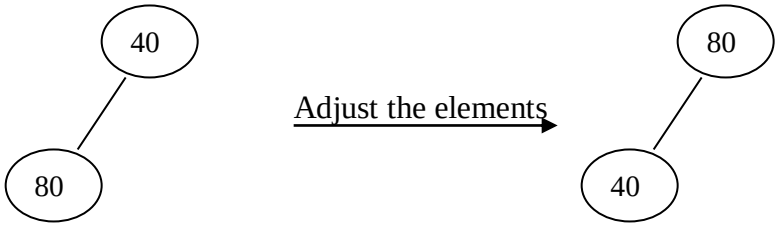
Example:

a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
40	80	35	90	45	50	70

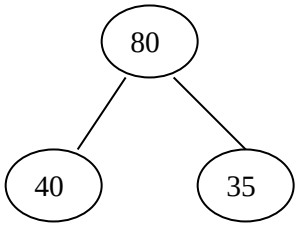
Step1:Take the first element of an array.a[1]=40.



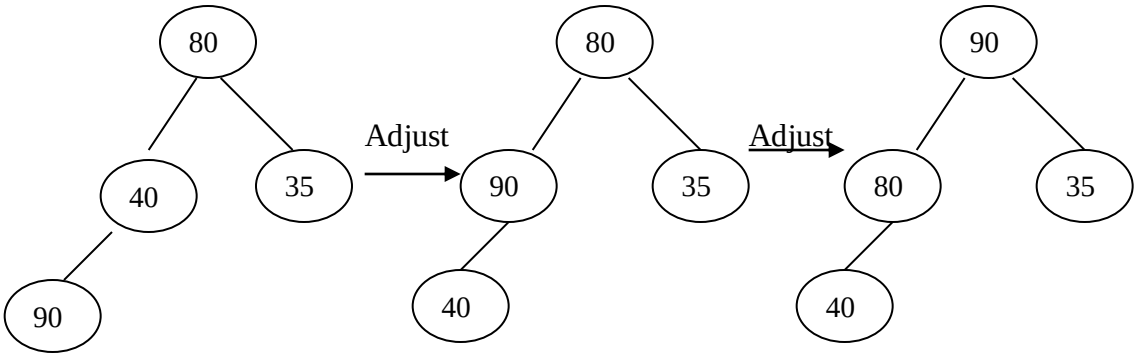
Step 2: Take the second element of an array.a[2]=80.
Insert a[2] to the left side of the first element.



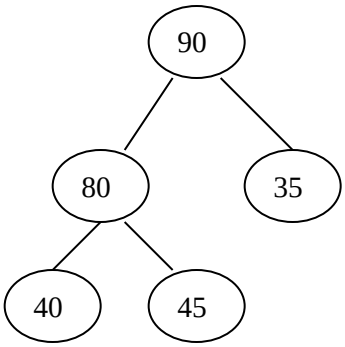
Step 3:Take the 3rd element .a[3]=35.Add this element to the right side of the root node.



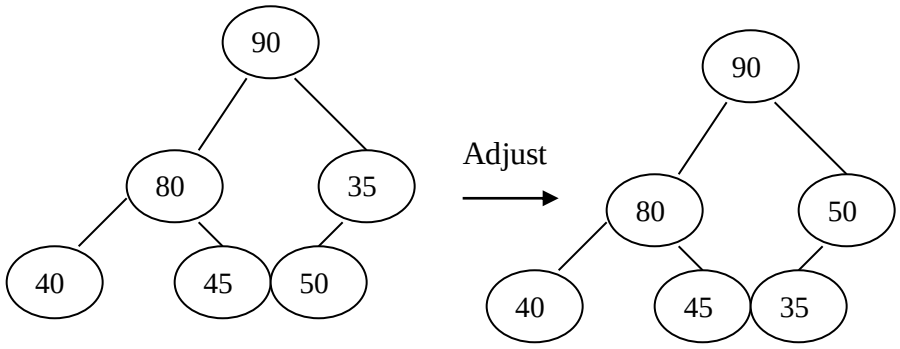
Step 4:Take the 4th element .a[4]=90.Add this element to the left side of the node 40.



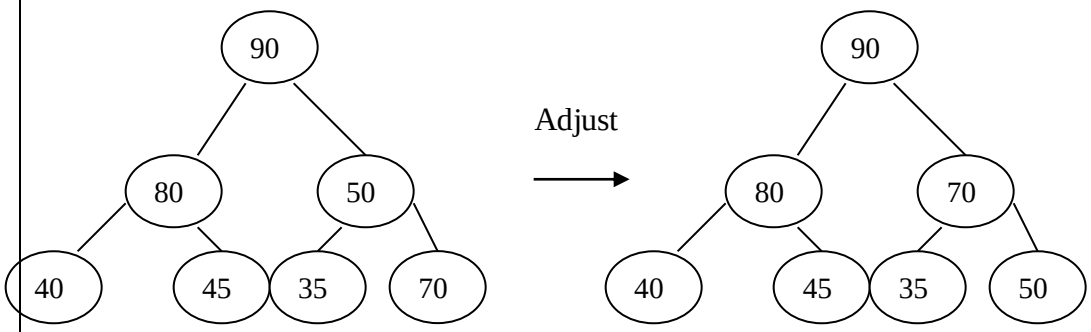
Step 5:Take the 5th element .a[5]=45.Add this element to the right side of the node 80.



Step 6:Take the 6th element .a[6]=50.Add this element to the left side of the node 35.



Step 7:Take the 7th element .a[7]=70.Add this element to the right side of the node 50.



The figure shows one example of how insert( ) would insert a new value into an existing heap.To do heap sort, first add the first element in the tree. Then add the second element to the tree.If the tree does not satisfy the heap property then adjust the nodes in the tree.Then insert the next element and adjust the nodes. Repeat the above steps till the insertion of last element. Finally we obtain the sorted elements in heap tree representation.

Analysis:

Heapify() requires n operations.->o(n)

Adjust() requires o(log n) operations for each invocation.Adjust is called n times by Heapsort( ).So time is o(n log n).

Heapsort = O(n) +O(n log n) =O(n).

Algorithm HeapSort( ):
T(n)=O(n)
Algorithm Heapify( ):
T(n)=O(n)
Algorithm Adjust( ):
T(n)=O(n log n)

TIME COMPLEXITY OF THE SEARCHING AND SORTING ALGORITHMS

Technique	Best case	Average case	Worst case
Searching			
1. Linear search	Ω (1)	θ (n)	O(n)
2. Binary search	Ω (1)	θ (logn)	O (logn)
Sorting			
1. Merge sort	Ω (nlogn)	θ (nlogn)	O(nlogn)
2. Quick sort	Ω (nlogn)	θ (nlogn)	O(n^2)
3. Heap sort	Ω (nlogn)	θ (nlogn)	O(nlogn)
4. Bubble sort	Ω (n)	θ (n^2)	O(n^2)
5. Selection sort	Ω (n^2)	θ (n^2)	O(n^2)
6. Insertion sort	Ω (n)	θ (n^2)	O(n^2)
7. Shell sort	Ω (nlogn)	θ (n^2)	O(n^2)
8. Radix sort	Ω (n)	θ (n)	O(n)

CST44 – DESIGN AND ANALYSIS OF ALGORITHMS
UNIT-1 TWO MARKS

1. Define Algorithm.

- Algorithm is a sequence of unambiguous instructions to solve a problem.
- A problem may have many solutions/algorithm. We need to select the best algorithm which is suitable for a given problem.
- After the algorithm for the problem is chosen, it is converted to program.

2. What are the properties of algorithm ?

- **Input:** Algorithm may take zero or more input
- **Output:** Algorithm always produces a single output
- **Definiteness:** Every instruction in an algorithm must be clear and unambiguous
- **Finiteness :** For all possible inputs, algorithm must produce result in finite steps
- **Effectiveness:** The algorithm must be feasible

3. List out the phases of program creation?

- Requirements
- Design
- Analysis
- Refinement and Coding
- Verification

4. What are the basic operations of data structures?

- Traversing
- Searching
- Sorting
- Insertion
- Deletion
- Merging

5. State the need for algorithm analysis

- Since a problem may have many algorithms, Comparing one algorithm with another is performed based on the *amount of computing resources used by algorithm*. By analyzing the algorithm, best algorithm for the problem can be chosen based on *time complexity and the space complexity*.

6. What is an array?

- Array is a collection of elements of same data type that are stored under a common name.
- The elements of array are stored in contiguous memory location.

7. Why do we use a Multidimensional array?

- A multidimensional array can be useful to organize subgroups of data within an array. In addition to organizing data stored in elements of an array, a multidimensional array can store memory addresses of data in a pointer array and an array of pointers.

- Syntax:
  - datatype arrayname[size 1][size 2][size 3]……[size n];
- Example:
  - int a[4][4][4] ; where a is a three dimensional array of int type and can hold 64 elements.

8. What are the various algorithmic notations?

Name of the algorithm : Each algorithm is given by its name

Data required for the algorithm: Data required for executing an algorithm is given as arguments to the algorithm

Comments : It describes the data passed to the algorithm or the steps of an algorithm

Steps of the algorithm: The sequence of steps which is performed by the algorithm

***Read* =>** Used to read the values

***Write* =>** Used to display the values

9. Define time complexity with example.

- It is the total amount of time required by an algorithm to complete its execution. It is also called as running time or execution time of an algorithm
- Time complexity depends on input data, read and write speed of machine etc
- **Example:**

```
for i=1 to n
  X= x+1
end  for
```

Number of times loop executed = ***n*** => **T(n) = O(n)**

10. What is the use of space complexity?

- It is the amount of memory resources utilized to execute an algorithm
- The memory is required to store program instructions, store variables, constants etc

11. Mention the use of pointer array.

- The Advantage of a pointer array is that, ***the pointers can be reordered in any manner without moving the data items.***
- By storing the address of the array element in a pointer, we can directly access that element in the array

12. What is an multidimensional array?

- An array with more than one subscript is called multi dimensional array.
- Multidimensional arrays are slower in execution than one dimensional arrays.

Example:

- int a[4][4][4] ; where a is a three dimensional array of int type and can hold 64 elements.

13. What do you meant by sorting?

- Sorting a sequence of elements involves ***rearranging them in either ascending or descending order*** depending upon the relationship among the elements.

14. What is the necessity for sorting technique?

- All data processing requires accessing records efficiently and quickly.
- Search techniques are most efficient only when the data items are sort according to some specified keys.
- If the list/file is not sorted, searching a record takes more time when the list/file is large.

15. What are the factors to be considered while choosing a sorting technique?

- Running time of the sorting technique
- Memory space needed for the sorting technique
- Number of comparisons required for sorting the list
- Programming time

16. What is meant by internal sorting?

- An internal sort is a sorting process that takes place entirely within the main memory of a computer.
- This is possible whenever the data to be sorted is small.
- For sorting larger data, it may be necessary to hold only small amount of data in memory at a time
- So, The rest of the data is normally held on some larger memory like a hard-disk.

17. When sorting method is said to be stable? Give example for stable and unstable sorting.

- A sorting method is said to be stable when it has minimum number of swaps
- **Stable sorting techniques:** Bubble sort, Insertion sort, Selection sort, Merge sort
- **Unstable sorting techniques:** Quick sort, Heap sort, Shell sort, Radix sort

18. What do you meant by searching?

- Searching is an operation to find the location of the given data in the array .
- Searching is said to be successful if the data is present otherwise said to be unsuccessful.
- There are three types of searching:
    - Linear search
    - Binary search
    - Fibonacci search

19. What is meant by linear search?

- Linear search is otherwise called as sequential search .
- Let a be the array of N numbers and X be the element to be searched .
- Our aim is to find whether X is present or not by comparing all the elements in array from 1 to N.

20. What is meant by binary search?

- Binary search is implemented using sorted array. It splits the array into two sub arrays based on middle element recursively.
- If the search element is a middle element, its position is returned or if it is less than the middle element, searching is performed on the left side of middle element, else, searching is performed on the right side of middle element.

21. What is meant by Fibonacci search ?

- The Fibonacci search technique is a method of searching a sorted array using divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers .
- Fibonacci search divides the array into two parts that have sizes that are consecutive Fibonacci numbers

22. What do you mean by performance analysis of an algorithm?

- It is a process of making evaluative judgement about the algorithms
- Performance analysis consider mainly execution speed, memory consumption of that algorithm
- Since there are multiple alternative algorithms will be available to solve a problem, we analyse and select a best suitable algorithm based on our requirements.

23. Compare linear search and binary search.

Linear Search	Binary Search
It is not as fast as binary search.	It is faster than linear search.
The elements need not be inputed in sorted order.	The elements must be inputed in sorted order.
Searching is performed in a sequential manner	Searching is performed based on the middle element in the array, recursively.
Time complexity is more	Time complexity is less

24. Define Big oh(O) notation

Big Oh is defined as

The function f(n) = O(g(n)) if and only if there exists constants c and n_0, such that $f(n) \leq c * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm

Example:

$f(n) = 3n + 2 \leq 4*n$ for all $n \geq 2$, where f(n) = 3n+2, c = 4 , g(n) =n and $n_0 = 2$

25. Define Omega(Ω) notation

Omega is defined as

The function f(n) = Ω(g(n)) if and only if there exists constants c and n_0, such that $f(n) \geq c * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm

Example:

$f(n) = 3n + 2 \geq 3*n$ for all $n \geq 1$, where f(n) = 3n+2, c = 3 , g(n) =n and $n_0 = 1$

26. Define Theta(Θ) notation.

Theta is defined as

The function f(n) = Θ(g(n)) if and only if there exists constants c1, c2 and n_0, such that $c1 * g(n) \leq f(n) \leq c2 * g(n)$ for all $n \geq n_0$. Here f(n) and g(n) are computation time of the algorithm

Example:

$3*n \leq 3n + 2 \leq 4*n$ for all $n \geq 2$, where c1=3, c2=4, g(n) =n, f(n) = 3n+2, $n_0 = 2$

27. Define heap.

- A heap is defined to be a complete binary tree with the property such that each parent node is larger than its child nodes.
- The root node in heap is the largest element in the tree.

28. Define max heap.

- A max heap referred as descending heap of size n is defined as a complete binary tree of n nodes such that the data in the each parent node is greater than its child nodes.
- The root node in heap is the largest element in the tree.

29. Define min heap.

- A min heap referred as ascending heap of size n is defined as a complete binary tree of n nodes such that the data in the each parent node is lesser than its child nodes.
- The root node in heap is the smallest element in the tree.

30. How sorting is performed in heap sort?

Step1: Set of elements in array is adjusted according to the max heap property.
Step2: Then the root element is swapped with the last element. Then the last element is deleted from heap and stored in array.
Step3: Then, Repeat the **step 1** until heap has no element

31. Which is the fastest sorting technique & why?

- Quick sort technique is the fastest sorting technique.
- The advantage is that, The data can be moved to great distance in one move to place it in its exact position as in final list which reduces unnecessary swaps.

32. Which is the fastest sorting technique & why?

- It follows divide and conquer technique.
- It divides the array into sub-array based on mid position recursively, until the we get a single element.
- It, then conquers each sub-array by merging each sub-array recursively thereby forming the sorted array.